Evaluación Integral de Minería de Datos y Machine Learning

Ecosistema analítico de scouting deportivo con Random Forest, FastAPI y aplicación web

Jair Benítez Oliva

Kenneth David Genes Díaz

Electiva II

Universidad Cooperativa de Colombia

Facultad de Ingeniería de Sistemas

11 de noviembre de 2026

Tabla de contenido

1. Introducción y objetivos....3
2. Marco teórico y glosario....3
3. Metodología y construcción del sistema....5
4. Análisis de resultados del código y experimentos....6
    4.1 Diagnóstico de overfitting y comparación inicial....6
    4.2 Evaluación del ensamble y curva ROC-AUC....7
    4.3 Interpretación de importancia de variables....9
5. Arquitectura de despliegue y evidencias de funcionamiento....10
6. Conclusiones y recomendaciones de negocio....12
Referencias....13

1. Introducción y objetivos

El presente informe documenta el desarrollo de un ecosistema analítico End-to-End orientado al scouting deportivo. El proyecto integra la extracción de datos desde MySQL, el preprocesamiento de variables, el entrenamiento y optimización de un modelo Random Forest, la serialización del modelo en formato Joblib, el despliegue de una API REST mediante FastAPI y el consumo de la predicción a través de una interfaz web interactiva. La solución se diseñó con base en la lógica de la metodología CRISP-DM, la cual organiza el trabajo en comprensión del problema, preparación de los datos, modelado, evaluación y despliegue.

El objetivo general consiste en construir un sistema que ayude a clasificar futbolistas en dos grupos: Élite y Estándar, utilizando como criterio de referencia el percentil 75 del valor de mercado. Como objetivos específicos se plantearon: (a) preparar un conjunto de datos limpio a partir de una fuente relacional, (b) comparar modelos para diagnosticar sobreajuste y paradoja de la exactitud, (c) optimizar un RandomForestClassifier con GridSearchCV, (d) desplegar el modelo mediante una API REST, y (e) ofrecer una interfaz sencilla para evaluar perfiles nuevos.

2. Marco teórico y glosario

Imputación por mediana agrupada. Técnica de tratamiento de valores faltantes en la cual los nulos se reemplazan por la mediana de un subgrupo, en este caso la posición del jugador, con el fin de reducir el efecto de valores extremos.

Target binario por percentil (Top 25%). Transformación de la variable continua valor de mercado en una etiqueta binaria: 1 para jugadores con valor mayor o igual al percentil 75 y 0 para los demás.

One-Hot Encoding. Conversión de variables categóricas nominales en columnas binarias para que los algoritmos basados en matrices puedan procesarlas.

Muestreo estratificado. Estrategia de división train/test que conserva la proporción original de la clase objetivo en ambos subconjuntos.

Sobreajuste. Fenómeno en el que un modelo aprende en exceso los patrones del conjunto de entrenamiento y pierde capacidad de generalización sobre datos no vistos.

Poda previa. Mecanismo de regularización en árboles de decisión que limita parámetros como profundidad o mínimo de muestras por hoja para controlar su complejidad.

Paradoja de la exactitud. Situación propia de conjuntos desbalanceados en la que un modelo puede tener accuracy aceptable aun cuando falla en la clase minoritaria de interés.

class_weight = balanced. Técnica que asigna mayor peso a la clase minoritaria para penalizar más los errores cometidos sobre ella durante el entrenamiento.

Random Forest y bagging. Modelo de ensamble que combina múltiples árboles entrenados sobre muestras bootstrap y subconjuntos aleatorios de variables, reduciendo la varianza del modelo.

GridSearchCV. Búsqueda exhaustiva sobre una malla de hiperparámetros utilizando validación cruzada para seleccionar la configuración con mejor rendimiento.

Curva ROC y AUC. Herramienta de evaluación que relaciona la tasa de verdaderos positivos y la tasa de falsos positivos a través de distintos umbrales; el AUC resume la capacidad discriminante global.

Importancia de variables. Medida derivada de la reducción de impureza total del bosque, que permite estimar el aporte relativo de cada predictor.

Serialización con Joblib. Proceso de guardar en disco un objeto de Python, como el pipeline de un modelo, para reutilizarlo posteriormente sin reentrenarlo.

API REST y arquitectura cliente-servidor. Modelo de interacción en el que el cliente envía solicitudes HTTP al servidor y recibe respuestas estructuradas, normalmente en formato JSON.

3. Metodología y construcción del sistema

La etapa de ingesta partió de la base de datos MySQL transfermarkt. alojada en XAMPP. La tabla players almacenó 50.149 registros originales. Luego de filtrar observaciones sin la información indispensable para el análisis. el conjunto válido quedó conformado por 41.495 jugadores. En este subconjunto. se identificaron 2.183 valores faltantes en estatura. los cuales fueron imputados mediante la mediana por posición. Para garantizar reproducibilidad. el cálculo de la edad se realizó con fecha de referencia 2026-10-07.

Posteriormente se construyó la variable objetivo Élite a partir del percentil 75 del valor de mercado. lo que fijó un umbral de €700.000. El conjunto quedó desbalanceado. con 11.077 casos Élite (26.69%) y 30.418 casos Estándar (73.31%). Después del One-Hot Encoding sobre la variable posición. se dividieron los datos con muestreo estratificado en 33.196 observaciones para entrenamiento y 8.299 para prueba.

En la fase de modelado se entrenaron dos árboles de decisión de referencia: un árbol libre y un árbol podado. Más adelante se entrenó un Random Forest con búsqueda combinatoria de hiperparámetros. La configuración ganadora quedó determinada por class_weight = balanced, max_depth = 10, min_samples_leaf = 15 y n_estimators = 80, con un F1 promedio de validación cruzada de 0.5132.

4. Análisis de resultados del código y experimentos

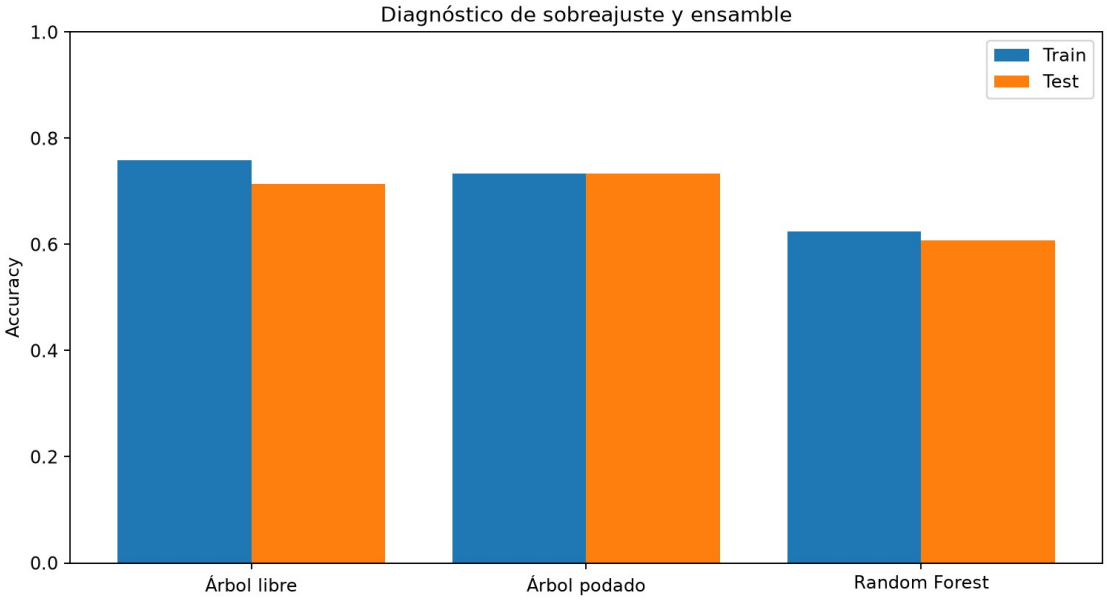
4.1 Diagnóstico de overfitting y comparación inicial

La primera comparación se realizó entre el árbol libre y el árbol podado. El árbol libre obtuvo mayor rendimiento en entrenamiento que en prueba, lo cual sugiere una tendencia al sobreajuste. Por su parte, el árbol podado redujo la brecha entre ambos conjuntos, aunque a costa de sacrificar la capacidad para detectar la clase Élite. Este comportamiento ilustra que controlar la complejidad del árbol mejora la generalización aparente, pero puede llevar a una subrepresentación de la clase minoritaria cuando el conjunto está desbalanceado.

Tabla 1. *Comparación de accuracy entre entrenamiento y prueba para el diagnóstico de sobreajuste*

Modelo	Accuracy train	Accuracy test
Árbol libre	75.83%	71.43%
Árbol podado	73.30%	73.31%
Random Forest	62.41%	60.74%

Figura 1. *Comparación de accuracy en train y test entre los modelos evaluados.*



Nota. La diferencia entre entrenamiento y prueba ayuda a identificar sobreajuste. El árbol libre presenta una brecha mayor que el árbol podado.

La llamada paradoja de la exactitud también quedó evidenciada. Aunque el árbol podado alcanzó un accuracy de 73.31% en prueba, su recall para la clase Élite fue 0.00%, es decir, no identificó correctamente a ningún jugador de la clase objetivo. Por tanto, el accuracy no fue suficiente para seleccionar el mejor modelo del proyecto.

4.2 Evaluación del ensamble y curva ROC-AUC

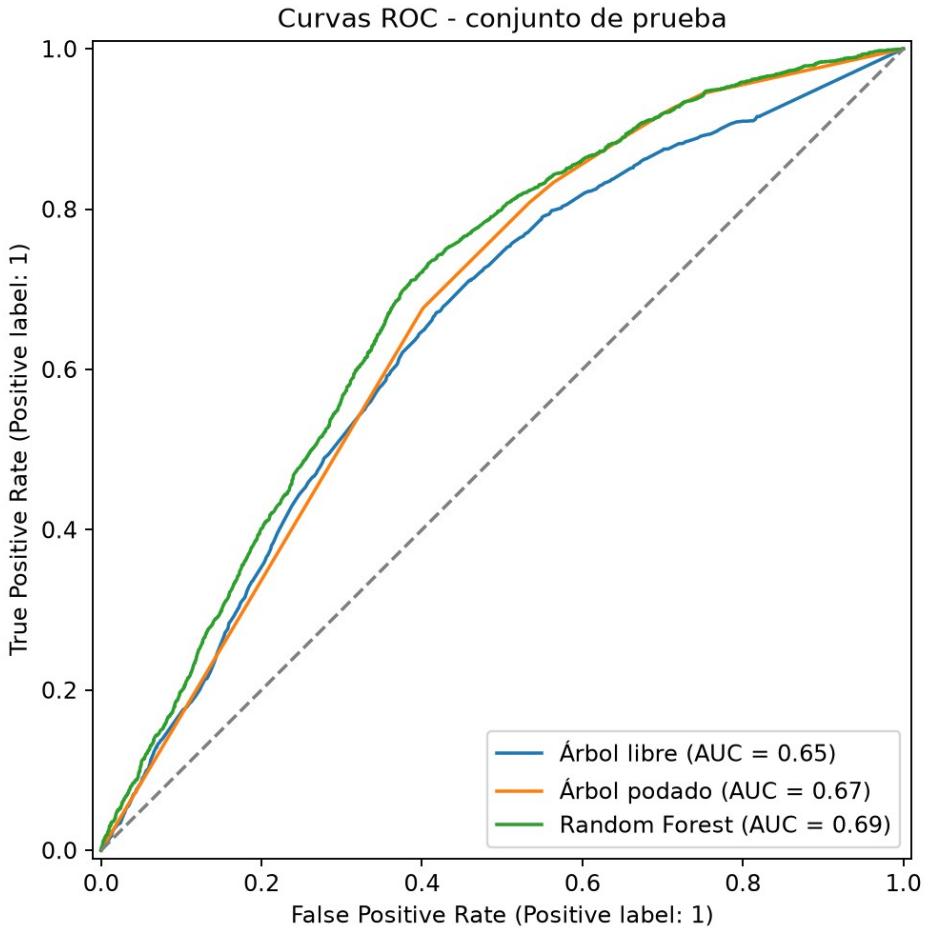
El Random Forest mostró el mejor equilibrio para la clase de interés. En prueba alcanzó accuracy de 60.74%, precisión de 38.20%, recall de 76.21%, F1 de 0.5089 y AUC de 0.6923. Estos resultados indican que el modelo sacrificó exactitud global para incrementar de forma importante la detección de futbolistas Élite. Dado que el objetivo del scouting consiste en reducir el riesgo de dejar pasar perfiles de alto valor, el recall y el F1 se priorizaron sobre el accuracy.

Tabla 2. *Métricas de evaluación en el conjunto de prueba*

Modelo	Accuracy	Precisión	Recall	F1	AUC
Árbol libre	71.43%	39.84%	13.81%	0.2052	0.6498
Árbol podado	73.31%	0.00%	0.00%	0.0000	0.6677
Random Forest	60.74%	38.20%	76.21%	0.5089	0.6923

A partir de la matriz de confusión del Random Forest se identificaron 1.688 verdaderos positivos, 527 falsos negativos, 3.353 verdaderos negativos y 2.731 falsos positivos. En otras palabras, el ensamble logró recuperar gran parte de los futbolistas de élite, aunque aún produce falsas alarmas que justifican la revisión humana posterior.

Figura 2. *Curvas ROC comparativas entre árbol de decisión y Random Forest.*

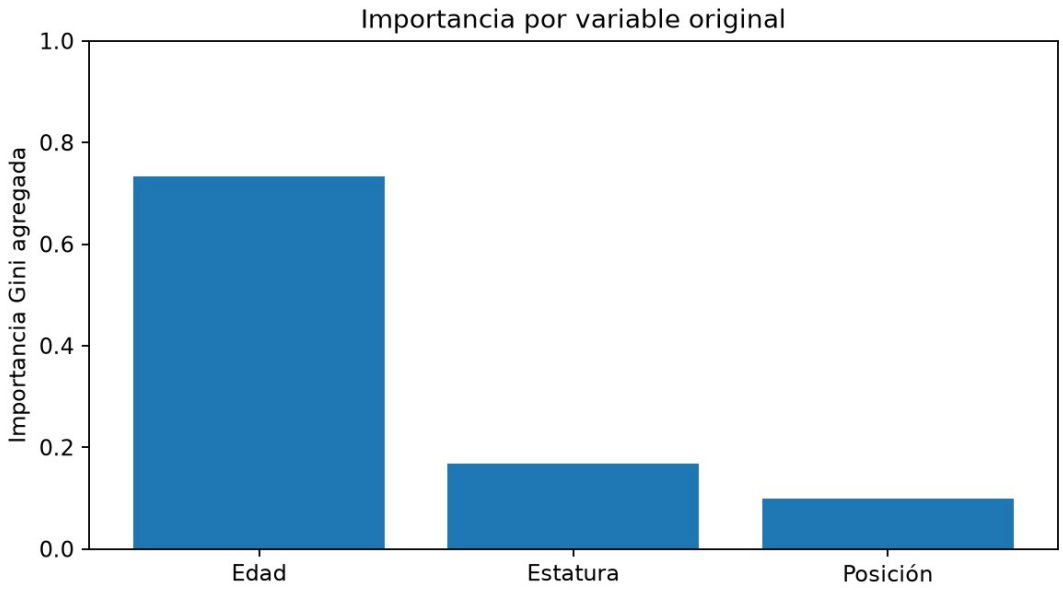


Nota. El área bajo la curva del Random Forest supera a la del árbol único, lo que confirma un mejor poder discriminante para separar futbolistas Élite y Estándar.

4.3 Interpretación de importancia de variables

El análisis de importancia de variables muestra que la edad concentra el mayor peso del modelo con 73.23%, seguida por la estatura con 16.79% y la posición con 9.98%. La lectura sustantiva de este resultado es coherente con el mercado deportivo: la edad suele reflejar cercanía al pico competitivo y potencial de valorización; la estatura aporta información relevante para roles específicos; y la posición, aunque importante, opera de forma más moderada cuando se combina con las variables anteriores.

Figura 3. *Importancia relativa de edad, estatura y posición en el Random Forest.*



Nota. La edad domina la reducción de impureza del bosque, por lo que actúa como principal variable explicativa en la clasificación del modelo.

5. Arquitectura de despliegue y evidencias de funcionamiento

El sistema final se estructuró bajo una arquitectura cliente-servidor. MySQL almacena la información histórica de los futbolistas; Python ejecuta la ingesta, el preprocesamiento y el entrenamiento del modelo; Joblib conserva el pipeline en disco; FastAPI expone los endpoints /health, /opciones y /predict; finalmente, el archivo index.html actúa como cliente web y consume la predicción mediante fetch y objetos JSON.

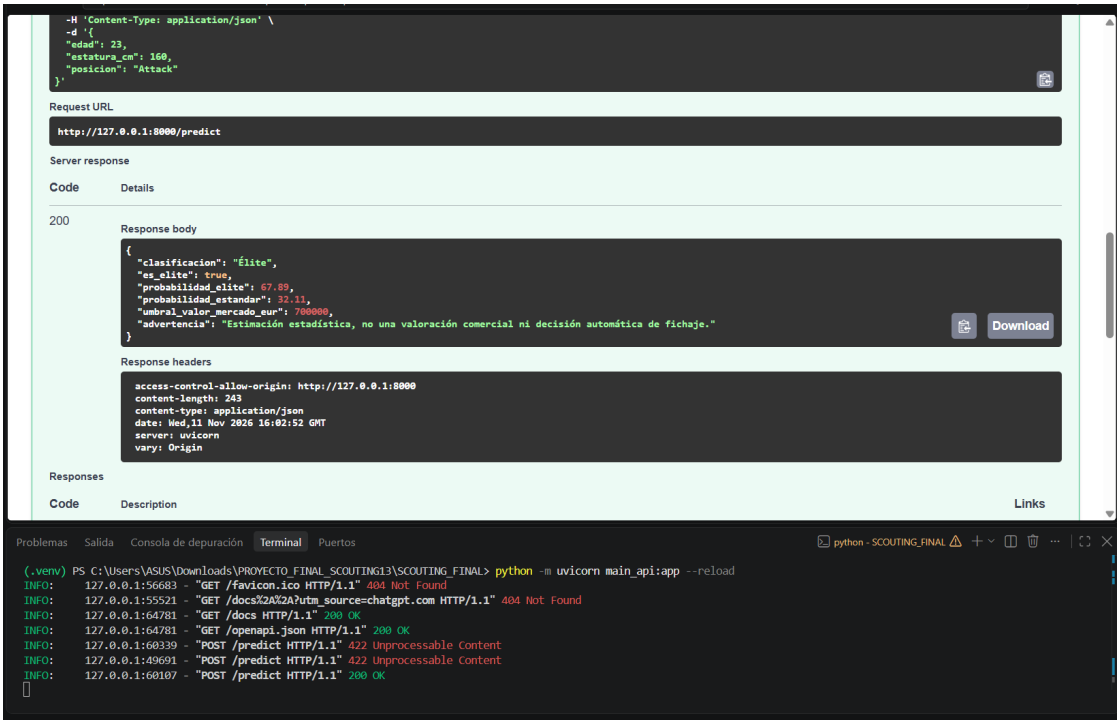
Figura 4. *Diagrama de arquitectura del ecosistema analítico implementado.*



Nota. El flujo inicia en la base de datos relacional, pasa por el módulo de entrenamiento y termina en la interfaz web que consulta la API REST.

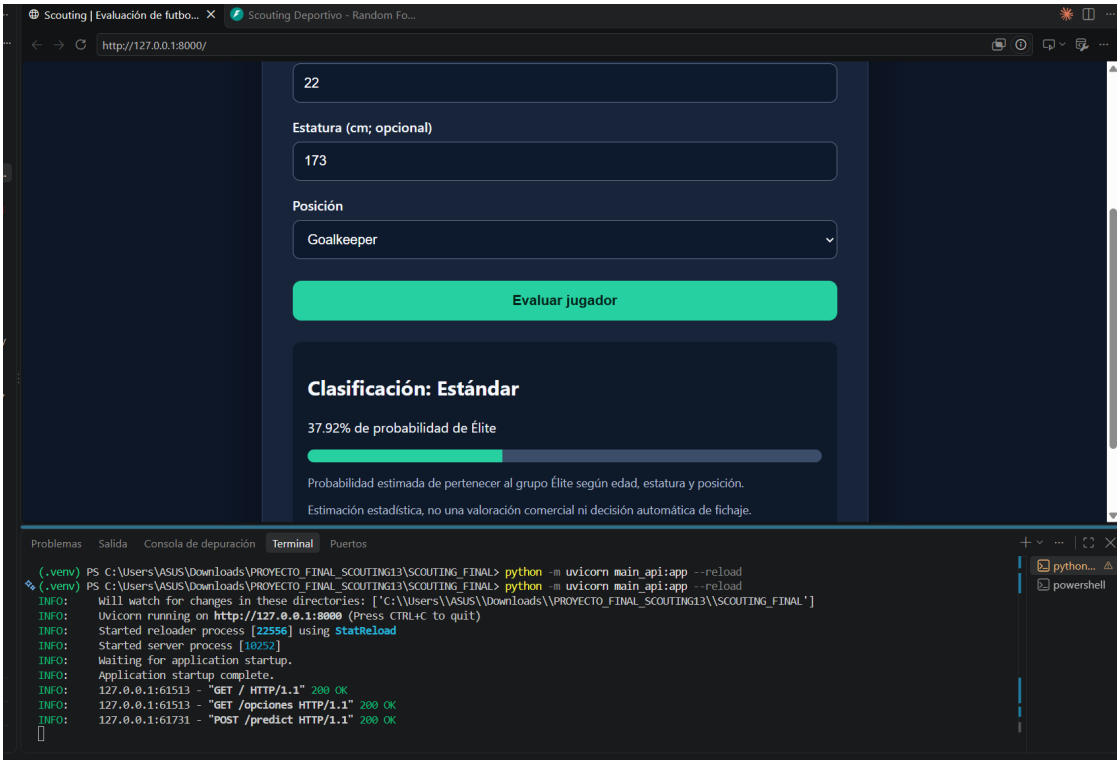
Durante la verificación en el equipo local se comprobó que la API respondió correctamente a consultas manuales y que el aplicativo web mostró predicciones para jugadores nuevos. Estas evidencias son importantes porque demuestran no solo el modelado, sino el despliegue funcional solicitado por la actividad.

Figura 5. *API REST en ejecución mediante Swagger con respuesta satisfactoria del endpoint POST /predict.*



Nota. La respuesta HTTP 200 OK confirma que FastAPI recibe, valida y procesa la información enviada al modelo.

Figura 6. *Aplicativo web respondiendo a la evaluación de un jugador nuevo.*



Nota. La interfaz consulta la API y presenta la clasificación estimada con su probabilidad correspondiente.

6. Conclusiones y recomendaciones de negocio

En primer lugar, se logró construir un ecosistema analítico completo desde la extracción de datos en MySQL hasta la visualización de la predicción en una interfaz web. Esto demuestra la integración exitosa entre base de datos relacional, Machine Learning, serialización del modelo y despliegue como servicio.

En segundo lugar, el diagnóstico de overfitting y la paradoja de la exactitud evidenciaron que el accuracy, por sí solo, no era un criterio suficiente para escoger el modelo. Aunque el árbol podado mostró una exactitud superior, su desempeño sobre la clase Élite fue nulo. En contraste, el Random Forest obtuvo el mejor equilibrio al alcanzar un recall de 76.21% y un F1 de 0.5089.

En tercer lugar, la edad se consolidó como el principal factor explicativo del modelo, seguida de la estatura y la posición. Este hallazgo sugiere que una política de scouting apoyada por analítica puede priorizar perfiles con mayor probabilidad de valorización, aunque siempre complementando el análisis con observación técnica, contexto competitivo y seguimiento longitudinal.

Como recomendación de negocio, el sistema puede usarse como herramienta de filtrado inicial para reducir el universo de candidatos y concentrar la atención del cuerpo de scouting en perfiles con alta probabilidad de pertenecer al segmento Élite. En futuras iteraciones convendría incorporar variables adicionales, como nacionalidad, pierna hábil, minutos jugados, estadísticas deportivas o historial de lesiones, con el fin de enriquecer la capacidad predictiva y disminuir la tasa de falsos positivos.

Referencias

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Han, J., Kamber, M., & Pei, J. (2012). Data mining: Concepts and techniques (3rd ed.). Morgan Kaufmann.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). An introduction to statistical learning: With applications in R (2nd ed.). Springer. https://doi.org/10.1007/978-1-0716-1418-1

Kuhn, M., & Johnson, K. (2019). Feature engineering and selection: A practical approach for predictive models. CRC Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,

Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Ramírez, S. (2026). FastAPI documentation. https://fastapi.tiangolo.com/